

DOCUMENTACIÓN TÉCNICA & ARQUITECTURA

Red de Salud – Guía de Algoritmos en Grafo

Este documento detalla la estructura interna, la ubicación en código y el flujo de ejecución de los algoritmos de optimización de redes implementados en el sistema informático de la **Red de Salud**.

1. Arquitectura General y Estructuras de Datos

La aplicación modela la infraestructura de distribución médica mediante un grafo ponderado compuesto por nodos (hospitales/hubs) y aristas (conexiones de transporte). Todos los datos primarios residen en dos variables globales:

1. Nodos: redSalud

Contiene los metadatos geográficos e institucionales de cada punto de la red.

```
redSalud = { 'LPZ': { nombre: 'La Paz - CEASS', tipo: 'Hub CEASS', lat: -16.5000, lng: -68.1500 }, 'SCZ': { ... } };
```

2. Aristas: adyacencia

Matriz de adyacencia con los pesos base entre nodos conectados.

```
adyacencia = { 'LPZ': { 'EAL': 0.5, 'HCL': 0.3 }, 'EAL': { 'LPZ': 0.5, 'ORU': 3.0 }, 'ORU': { ... } };
```

2. Ubicación Exacta de los Algoritmos en Código

Todos los algoritmos de procesamiento están contenidos dentro del bloque principal de `<script>` ubicado en la sección inferior del documento HTML.

Algoritmo / Función	Línea Aprox.	Función Principal	Identificador Visual
<code>ejecutar()</code>	~1180	Despachador central y validador de entradas.	—
<code>runBFS(start)</code>	~1210	Búsqueda en Amplitud (recorrido por niveles).	Azul
<code>runDFS(start)</code>	~1270	Búsqueda en Profundidad (exploración recursiva).	Verde
<code>runDijkstra(start, target)</code>	~1330	Ruta mínima (camino de costo/tiempo óptimo).	Naranja
<code>runKruskal()</code>	~1400	Árbol de Expansión Mínimo global (MST).	Morado
<code>runPrim(start)</code>	~1470	Árbol de Expansión Mínimo local (MST).	Rojo

3. Despachador Principal: ejecutar()

Función de control invocada por la interfaz (Botón "Ejecutar" o atajo `Ctrl + E`). Determina el algoritmo seleccionado y coordina la limpieza y redibujado de la interfaz.

```
function ejecutar() { var algo = document.querySelector('input[name="algoritmo"]:checked').value; var src = document.getElementById('origen').value; var dst = document.getElementById('destino').value; if (!src) { mostrarToast('Seleccione ORIGEN', 'warning'); return; } // Limpieza de capas y llamadas condicionales if (algo === 'bfs') runBFS(src); else if (algo === 'dfs') runDFS(src); else if (algo === 'dijkstra') runDijkstra(src, dst); else if (algo === 'kruskal') runKruskal(); else if (algo === 'prim') runPrim(src); }
```

4. Detalle de Implementación de Algoritmos

BFS (Breadth-First Search)

Explora el grafo nivel por nivel usando una cola (FIFO). Ideal para modelar propagación concéntrica de insumos desde un Hub.

```
var visited = new Set([start]);
var queue = [[start]]; while
(queue.length > 0) { var path =
queue.shift(); var node =
path[path.length - 1]; // Explora
vecinos e incrementa niveles }
```

DFS (Depth-First Search)

Recorre ramas completas hacia la máxima profundidad antes de retroceder. Útil para verificar conectividad y rutas secundarias.

```
function dfs(node, profundidad)
{ visited.add(node);
Object.keys(adyacencia[node]||
{}).forEach(v => { if (!visited.has(v))
dfs(v, profundidad + 1); }); }
```

Dijkstra (Ruta Mínima)

Calcula el camino más corto entre origen y destino seleccionando iterativamente el nodo no visitado con menor distancia acumulada.

```
dist[start] = 0; while
(unvisited.size > 0) { var u =
minDistanceNode(unvisited, dist);
unvisited.delete(u); // Relaxación de
aristas adyacentes }
```

Kruskal & Prim (MST)

Kruskal: Ordena aristas globalmente y aplica *Union-Find*.

Prim: Crece desde un nodo inicial buscando la arista más barata conexa.

```
// Kruskal: Union-Find sin
ciclos function union(a, b) { ... } //
Prim: Arista mínima conexa al árbol
visited.forEach(u => findMinEdge(u));
```

5. Sistema de Ponderación Dinámica

El peso de cada arista no es estático; responde al criterio seleccionado en la UI mediante la función de cálculo multi-criterio:

```
pesosMultiples[key] = { tiempo: t, // Horas base distancia: Math.round(t * 60), //
Estimación en Km costo: Math.round(t * 60 * 0.8 + 20), // Costo operativo riesgo:
Math.min(10, Math.max(1, Math.round(t * 0.8 + 2))) // Índice de riesgo (1-10) };
```

6. Flujo de Ejecución del Sistema

- 1. Entrada de Datos:** Selección de dataset preconfigurado o archivo CSV → Llenado de redSalud y adyacencia.
- 2. Configuración UI:** Definición del criterio de ponderación (Tiempo/Distancia/Costo/Riesgo) y selección de nodos Origen/Destino.
- 3. Ejecución:** Disparo de ejecutar() → Selección del algoritmo objetivo.
- 4. Salida & Rendering:** Dibujado de trazados sobre mapa Leaflet + renderizado de gráficos comparativos e historial.